

Ejercicios Clases Kotlin

[Descargar estos ejercicios](#)

1. [Ejercicio 1](#)
2. [Ejercicio 2](#)
3. [Ejercicio 3](#)

Ejercicio 1

Composición: Vehículo y Motor

Implementa una relación de **composición** entre las clases **Vehículo** y **Motor** . El motor es parte integral del vehículo y no puede existir sin él.

Ejercicio 1: Composición Vehículo-Motor

Creando vehículo con motor integrado...

Vehículo: Toyota Corolla

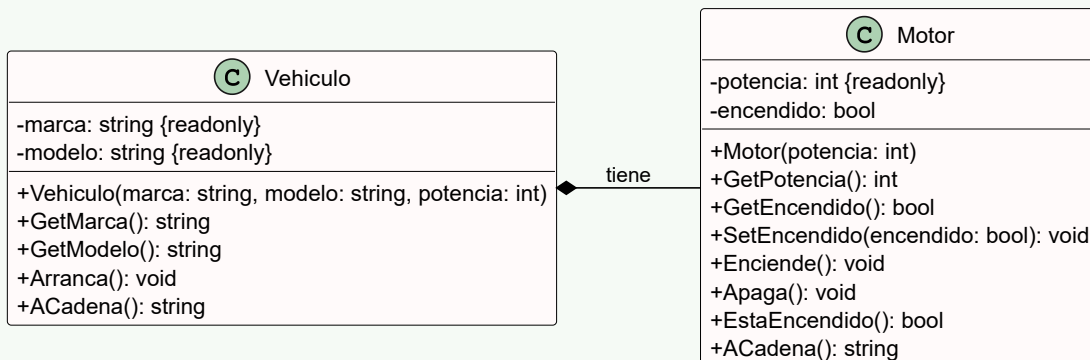
Motor: 120 CV - Estado: Apagado

Arrancando el vehículo...

Vehículo: Toyota Corolla

Motor: 120 CV - Estado: Encendido

Presiona cualquier tecla para salir...



Requisitos:

- Crea las clases necesarias para implementar correctamente el diagrama de clases. Sobre todo, fíjate en la relación entre clases y lo que conlleva.
- En el programa principal, crea las instancias de los objetos necesarias y Muestra el estado inicial como se ve en la salida
- Arrancar el vehículo y muestra el estado del vehículo después de arrancado.

Ejercicio 2

`data class` como Value Objects

Implementa un sistema que use dos tipos de `data class` como Value Objects: uno simplificado y otro extendido (con lógica en propiedades y/o constructores). El programa debe demostrar el uso de `vo.copy(..)` para crear copias modificadas en el `data class` simplificado, y la creación de nuevos objetos en el `data class` extendido.

Ejercicio 2: data class como Value Objects

```
=== DATA CLASS SIMPLIFICADOS (Value Objects sin validación) ===
Creando juguetes con data class simplificados...

Juguete original: Juguete(nombre=Pelota, precio=15.5)
Juguete con descuento usando '.copy(...)': Juguete(nombre=Pelota, precio=12.4)
Juguete renombrado usando '.copy(...)': Juguete(nombre=Pelota Roja, precio=15.5)
Juguete modificado completamente: Juguete(nombre=Mueca, precio=25.99)

--- Operaciones con juguetes ---
Precio promedio entre juguete original y con descuento: 13.95
Diferencia de precio: 3.1

=== DATA CLASS EXTENDIDOS (Value Objects con validación) ===
Creando temperaturas con data class extendidos y validación...

--- Creación exitosa ---
Temperatura ambiente: Temperatura(grados=22.5, escala=CELSIUS)
Temperatura agua hirviendo: Temperatura(grados=100.0, escala=CELSIUS)
Temperatura agua congelando: Temperatura(grados=32.0, escala=FAHRENHEIT)

--- Conversiones de temperatura (creando nuevos objetos) ---
22.5C convertido a Fahrenheit: Temperatura(grados=72.5, escala=FAHRENHEIT)
100C convertido a Kelvin: Temperatura(grados=373.15, escala=KELVIN)
32F convertido a Celsius: Temperatura(grados=0.0, escala=CELSIUS)
```

Requisitos:

- `data class` **simplificado Juguete**:
 - Crea el `data class` `Juguete` con las propiedades `nombre` de tipo `String` y `precio` de tipo `Double`.
 - `calculaDescuento(porcentaje : Double)` método que devuelve el `Juguete` con el precio modificado con el descuento.

- **data class extendido Temperatura:**

Este Value Object tendrá una propiedad de con el enumerado `EscalaTemperatura` con los valores (Celsius, Fahrenheit, Kelvin)

- **data class** `Temperatura` con las propiedades `grados` de tipo `double` y `escala` de tipo `EscalaTemperatura`.
- Constructor con validación (no permite temperaturas bajo cero absoluto). Validaremos según la información de la ayuda.
- Valores del cero absoluto según la escala:
 - **Celsius:** -273.15°C (cero absoluto)
 - **Fahrenheit:** -459.67°F (cero absoluto)
 - **Kelvin:** 0.0°K (cero absoluto)

Si intentas crear una temperatura inferior a estos valores, el constructor debe asignar automáticamente el valor del cero absoluto correspondiente a la escala.

- Métodos `convierteA(nueva: EscalaTemperatura)` que devolverá la nueva Temperatura calculada a partir de la escala que llega. ***Para realizar el cálculo, podría ser conveniente pasar los grados del objeto a Celsius para posteriormente pasarlos a la escala de temperatura nueva, se deberá tener en cuenta en que escala está el objeto.***

- **En el programa principal:**

- Tendremos el método `gestionJuguete`, que crea un juguetes base y se modifica con `with` para conseguir la salida. Además de calcular descuentos y comparar precios.
- Tendremos el método `gestionTemperatura` que creará temperaturas válidas e inválidas, convertirá entre escalas de temperatura.

Ayuda

Para que el constructor de la clase `Temperatura` pueda validar implementa un **interceptor del constructor** `operator fun invoke(...)` en el companion object de la clase `Temperatura` que reciba los grados y la escala, y devuelva un objeto Temperatura con la validación del cero absoluto. Por ejemplo:

```
operator fun invoke(  
    grados: Double,  
    escala: EscalaTemperatura): Temperatura {  
  
    val cero = ceroAbsoluto(  
        escala = escala  
    )  
    val valorFinal = max(  
        a = grados,  
        b = cero  
    )  
    return Temperatura(  
        grados = valorFinal,  
        escala = escala  
    )  
}
```

Ejercicio 3

Patrón Sum Type: Resultado de Pago

Estás desarrollando el **módulo de cobros de una tienda online**. Dependiendo del monto de la compra y del estado del sistema, un pago puede tener tres resultados posibles, y cada uno lleva información diferente.

Debes implementar el **patrón Sum Type** para modelar el resultado de una transacción y un objeto que procese dichos pagos.

Especificaciones:

1. Crea una sealed interface llamada `ResultadoPago`.
2. Definir las Variantes (Los Casos): Crea tres **data classes** anidadas que implementen

`ResultadoPago`:

- `Aprobado`: Debe contener un `UUID` `codigoTransaccion`.
- `Rechazado`: Debe contener un `String` `motivo`.
- `EnRevision`: (Para montos sospechosos) Debe contener un `Int` `nivelRiesgo` (ej. de 1 a 5).

3. Crea un `object` llamado `PasarelaDePago` con la función:

```
fun procesar(monto: BigDecimal): ResultadoPago
```

Implementa la siguiente lógica simulada:

- **Si el monto es menor a cero**: Retorna `Rechazado` con motivo `"Monto inválido"`.
- **Si el monto es mayor a 5000**: Retorna `EnRevision` con nivel de riesgo `5`.
- **Si el monto es mayor a 1000**: Retorna `EnRevision` con nivel de riesgo `1`.
- **En cualquier otro caso**: Retorna `Aprobado` con un identificador generado aleatoriamente (`UUID.randomUUID()`).

4. En la función `main`:

- Crea una lista inmutable (`listOf`) de montos de prueba (`BigDecimal`): -50, 100, 2500 y 6000.
- Itera sobre la lista procesando cada pago con `PasarelaDePago.procesar(monto)`.
- Utiliza una expresión `when` exhaustiva (Pattern Matching / Smart Casts) sobre el resultado para formatear e imprimir un mensaje en consola según el tipo, accediendo directamente a sus propiedades:
 - `ResultadoPago.Aprobado`:
`" [OK] Pago exitoso. ID: ${resultado.codigoTransaccion} "`
 - `ResultadoPago.Rechazado`: `" [X] Error en el pago. Motivo: ${resultado.motivo} "`

- **ResultadoPago.EnRevision :**

" [?] El pago requiere auditoría. Riesgo: \${resultado.nivelRiesgo} "

- Muestra por consola el monto formateado en moneda junto con el mensaje resultante.